Appendix

A. Training Protocol

Convolutional Neural Networks. For convolutional backbones (ResNet-50, ResNet-101, VGG-11, and VGG-19), we pretrain using Stochastic Gradient Descent (SGD) with a learning rate of 0.1. However, for VGG-19, the learning rate was reduced to 0.05 to prevent gradient explosion due to its depth. All models are finetuned using AdamW with a lower learning rate of 1e-4 for stable task adaptation. We prune at the beginning of each epoch for 5 epochs and allow 10 epochs for recovery post-pruning.

Table 1: BaCP Hyperparameters for Convolutional Neural Networks

Hyperparameter	**ResNet-50/101**	**VGG-11/19**
Dataset	CIFAR-10/100	
Pretraining Optimizer and LR	SGD(0.1)	SGD(0.1) / SGD(0.05)
Finetuning Optimizer and LR	AdamW(1e-4)	
Epochs	5	
Recovery Epochs	10	
Batch Size	512	

Vision Transformers. For vision transformers (ViT-Tiny and ViT-Small), We maintain the same pre-training and finetuning strategy used for CNNs. Both models are pretrained using SGD with a learning rate of 0.1 and finetuned using AdamW with a learning rate of 1e-4.

Table 2: BaCP Hyperparameters for Vision Transformers

Hyperparameter	**ViT-Tiny**	**ViT-S**
Dataset	CIFAR-10/100	
Pretraining Optimizer and LR	SGD(0.1)	
Finetuning Optimizer and LR	AdamW(1e-4)	
Epochs	5	
Recovery Epochs	10	
Batch Size	256	

Language Models. For DistilBERT and RoBERTa, we use a learning rate of 5e-5 and 1e-5, respectively, for pretraining on the SST-2 dataset. Both models are then finetuned using AdamW with a learning rate of 1e-4. When pretraining on the WikiText-2 dataset, we use AdamW with a learning rate of 1e-3 for DistilBERT and 5e-4 for RoBERTa.

Table 3: BaCP Hyperparameters for Language Models

Hyperparameter	**DistilBERT**	**RoBERTa**
Dataset	SST-2 / WikiText-2	
Pretraining Optimizer and LR	AdamW(5e-5) / AdamW(1e-3)	AdamW(1e-5) / AdamW(5e-4)
Finetuning Optimizer and LR	AdamW(1e-4)	
Batch Size	64	

B. Ablation Study

B.1 Multi-Objective Loss Weights To balance the contributions of each loss module, we begin by initializing the lambdas $\lambda_{1,2,3,4}$ to a static value of 0.25, effectively assigning equal importance to all four loss components. This static setting ensures uniform weighting throughout training, but it assumes all objectives are equally useful under all conditions, which may not be true under extreme sparsity constraints.

To introduce more flexibility, we also explore a learnable lambda configuration. In this setup, we initialize the lambdas as learnable parameters set to 0 which are passed through a softmax activation function to start with an effective value of 0.25. During training, these parameters are updated every step with a learning rate of 1e-4 and are re-normalized after each step to ensure they sum to 1. This learnable setup allows the model the prioritize the loss terms that are most beneficial for learning and convergence.

Table 4 compares the final lambda values of each lambda (after training) over three sparsity levels: 0.95, 0.97, and 0.99 as well as the Top-1 Accuracy.

Table 4: Impact of Static vs Learnable Lambda Parameters on Accuracy at Varying Sparsity Levels

Lambda Type	λ_1	λ_2	λ_3	λ_4	Accuracy (%) at Sparsity		
					0.95	0.97	0.99
Static (Current)	0.2500	0.2500	0.2500	0.2500	93.58	93.27	92.32
Learnable Parameters	0.9884	0.0030	0.0049	0.0036	93.24		
	0.9886	0.0030	0.0048	0.0035		93.11	
	0.9886	0.0030	0.0049	0.0036			91.93

Across all three sparsity levels, the model strongly favors Cross Entropy (CE) Loss in the learnable configuration. This is evident from the final lambda values, where $\lambda_1 \sim 0.99$ while the others shrink to zero. Despite the shift, accuracy drops slightly, indication that while CE is still a powerful loss function, an over-reliance on CE degrades performance at high sparsity levels, where contrastive learning helps retain richer representations. Below is the evolution of the learnable lambda parameters under the three sparsity constrains.

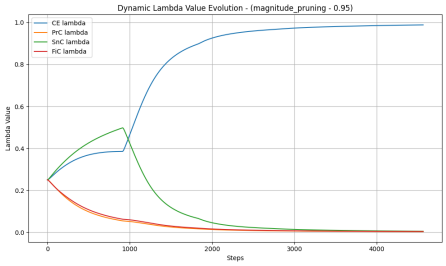

Figure 1: Sparsity at 0.95

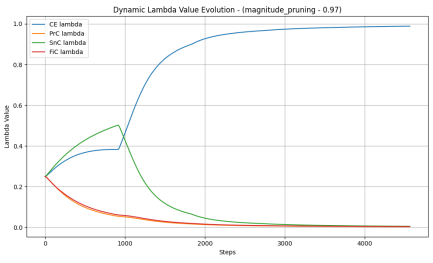

Figure 2: Sparsity at 0.97

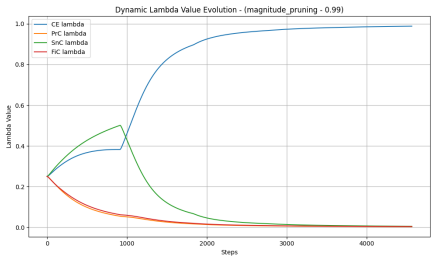

Figure 3: Sparsity at 0.99

B.2 Loss Module Importance Each loss module in the BaCP framework plays an important role in balancing multiple objectives to maintain representation quality, especially under high sparsity constraints. To evaluate the contribution of each module, we perform an ablation study by removing one component at a time from the total loss. This is achieved by setting the individual module's weight (λ_i) to zero, effectively excluding it from the training objective. We then observe the effect on top-1 accuracy across three sparsity levels: 0.95, 0.97, and 0.99. The results are summarized in Table 5.

Table 5: (Effect of Loss Module on Accuracy at Different Sparsity Levels ($\lambda_i = 0$).

λ_1 (CE)	λ_2 (PrC)	λ_2 (SnC)	λ_2 (FiC)	Modules Used	Accuracy (%) at Sparsity		
					0.95	0.97	0.99
✓	✓	✓	✓	BaCP (Current)	93.58	93.27	92.32
×	✓	✓	✓	no CE	93.59	93.40	92.54
✓	×	✓	✓	no PrC	93.52	92.99	92.68
✓	✓	×	✓	no SnC	92.98	92.99	92.55
✓	✓	✓	×	no FiC	92.79	93.27	92.15

In our framework, each loss module is weighted by either a learnable or fixed lambda coefficient. The coefficients are normalized in a manner that their total sum $\Sigma_{i=1}^{4}\lambda_i = 1$. When once lambda is zeroed out, the remaining three are re-normalized to maintain a balanced contribution. This allows us to analyze the individual loss terms without having to skew the total loss magnitude.

B.3 Supervised vs. Self-Supervised Contrastive Loss Contribution To further examine the effective of different loss components in BaCP, we investigate the individual and combined contributions of supervised and self-supervised contrastive losses. Each experiment selectively disables one of both of the loss types to observe their impact on the Top-1 Accuracy under varying sparsity levels. The results of this experiment are presented in Table 6

Table 6: Effect of Contrastive Learning Type on Accuracy at Different Sparsity Levels.

Supervised Contrastive	Self-Supervised Contrastive	Accuracy (%) at Sparsity		
		0.95	0.97	0.99
✓	✓	93.58	93.27	92.32
✓	×	93.79	93.09	92.66
×	✓	92.96	93.36	92.96
×	×	93.48	93.31	92.63

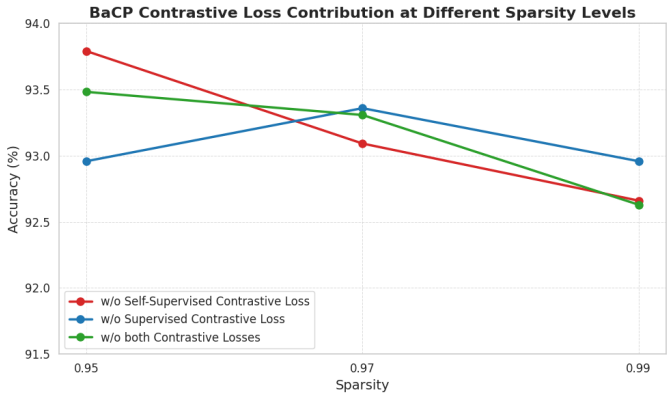

Figure 4: Effect of removing either supervised or self-supervised contrastive loss.

B.4 Temperature Sensitivity To determine the optimal value of tau for BaCO, we run a temperature sweep and report the Top-1 Accuracy over three sparsity levels: 0.95, 0.97, and 0.99. Table 7 summarizes the results on CIFAR-10 with ResNet-50. All configurations are held constant (optimizer, learning rate, batch size, and epochs) as defined in Table 1.

All runs use the same configuration (optimizer, learning rate, batch size, epochs) as outlined in Table 1.

Table 7: Effect of Temperature τ on Accuracy (%) across Target Sparsity Levels.

τ	Sparsity 0.95	Sparsity 0.97	Sparsity 0.99
0.07	93.55	93.31	92.48
0.15 (current)	93.58	93.27	92.32
0.2	93.54	93.65	92.02
0.3	92.94	93.02	91.97
0.5	92.91	92.94	92.31
0.7	92.91	92.73	91.95
1.0	92.93	92.71	92.22

Figure 5 below visualizes the effect of τ on accuracy. The accuracy peaks around $\tau = 0.07$–0.2, confirming the sweet spot for contrastive supervision in sparse regimes.

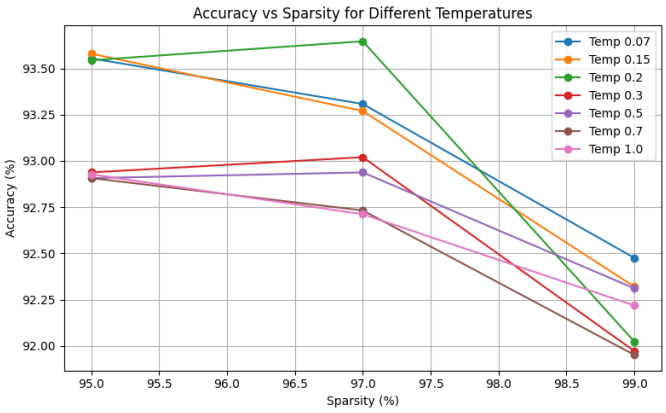

Figure 5: Temperature sensitivity plot showing accuracy vs. τ for different sparsity levels.

B.5 Ablation Test on All Datasets To verify the scalability of BaCP, we test it on multiple different datasets. All experiments utilized training protocol 1. However, ViT's used a learning rate of 0.01 instead of 0.1. The results are displayed in Table 8.

Table 8: Testing BaCP across datasets using Magnitude and SNIP-it pruning at varying sparsity levels.

Model	Dataset	Dense	95% Sparsity				97% Sparsity				99% Sparsity			
			Mag		SNIP-it		Mag		SNIP-it		Mag		SNIP-it	
			I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP
ResNet-50	MNIST	99.41	99.53	99.61	99.46	99.64	99.56	99.63	99.60	99.63	99.47	99.59	99.44	99.55
	FMNIST	93.63	93.32	94.22	93.16	94.15	93.02	94.54	92.88	94.09	92.28	94.43	92.87	94.47
	EMNIST (Balanced)	89.24	88.79	90.51	88.94	90.64	88.92	90.56	89.31	90.41	89.06	90.51	89.64	90.53
	SVHN	96.40	95.88	96.95	95.50	96.98	95.60	92.02	95.60	97.32	95.44	97.04	95.34	96.75
ResNet-101	MNIST	99.50	99.50	99.60	99.52	99.58	99.54	99.54	99.58	99.56	99.42	99.66	99.53	99.59
	FMNIST	93.30	93.27	94.43	92.74	94.39	93.39	94.44	92.76	94.38	92.82	94.40	92.52	94.27
	EMNIST (Balanced)	89.11	89.02	90.25	88.72	90.28	88.61	90.13	88.51	90.56	89.27	90.55	89.10	90.42
	SVHN	96.45	95.54	96.95	95.51	97.00	95.57	96.99	95.46	97.14	95.44	96.81	94.83	97.01
VGG-11	MNIST	99.56	99.52	99.55	99.61	99.53	99.54	99.56	99.62	99.59	99.58	99.51	99.55	99.52
	FMNIST	93.85	93.29	93.42	93.32	93.26	93.48	93.42	93.18	93.09	93.27	93.17	92.71	92.91
	EMNIST (Balanced)	89.75	89.18	90.17	89.54	90.02	89.24	90.10	89.69	90.01	89.60	90.03	89.00	89.97
	SVHN	95.14	95.10	96.43	95.29	96.20	95.08	96.36	95.12	96.16	95.30	96.33	95.45	96.00
VGG-19	MNIST	99.66	99.69	99.66	99.59	99.54	99.63	99.62	99.01	99.63	98.56	99.64	99.54	99.70
	FMNIST	93.91												
	EMNIST (Balanced)	90.21	89.12	90.55	89.67	89.96	89.26	90.09	89.73	89.97	90.01	90.39	85.53	90.17
	SVHN	96.24	96.12	96.68	96.31	96.38	95.80	96.89	95.88	96.64	95.79	96.96	95.93	96.87
ViT-Tiny	MNIST													
	FMNIST													
	EMNIST (Balanced)	90.80	88.47	89.68	88.73	89.22	88.32	89.68	87.91	89.54	87.98	Y 89.35	87.07	89.18
	SVHN	96.86	94.84	96.63	94.50	95.83	94.14	96.04	94.39	95.83	89.45	95.80	93.75	95.29
ViT-Small	MNIST													
	FMNIST													
	EMNIST (Balanced)	90.87	88.81	90.05	89.00	89.95	89.11	89.98	88.60	89.93	88.23	89.64	88.69	89.53
	SVHN	97.31	93.75	96.32	94.81	96.79	93.43	96.24	94.96	96.79	75.76	95.49	94.40	96.26

C. WikiText-2 Perplexity

Perplexity is a standard metric used to evaluate language models on generative tasks. It quantifies how well a model predicts the next token in a sequence with lower values indicating high levels of confidence. Specifically, perplexity is computed as the exponential of the average negative log-likelihood loss over the predicted tokens.

For decoder-only transformer models such as DistilBERT and RoBERTa applied to masked language modeling (MLM) tasks, lower perplexity implies less uncertainty and greater confidence in the model's prediction.

Below, we report the perplexity of RoBERTa and DistilBERT on the WikiText-2 dataset across various sparsity levels and pruning strategies:

Table 9: Perplexity Comparison at Different Sparsity Levels Across Models, Pruning Strategies, and BaCP Integration on WikiText-2 (Lower is Better)

Model	Dataset	Dense	95% Sparsity						97% Sparsity						99% Sparsity					
			Magnitude		SNIP-it		WANDA		Magnitude		SNIP-it		WANDA		Magnitude		SNIP-it		WANDA	
			I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP
DistilBERT	WikiText-2	5.743	27.106	20.780	33.934	28.470	33.869	27.041	30.643	28.064	40.321	30.940	38.139	31.885	45.728	37.505	64.528	51.993	48.597	40.529
RoBERTa	WikiText-2	3.508	15.584	14.267	16.637	16.516	18.401	17.317	18.655	16.023	19.555	22.327	267.192	22.896	28.423	23.810	30.567	32.496	662.66	29.839

D. Code Algorithms

D.1 Pruning Base Class We define an abstract base class `Pruner` which outlines the key interface methods for all pruning strategies. This includes:

- Maintaining and applying masks to the model's weights.
- Scheduling sparsity across epochs (supports both linear and cubic schedulers).
- Setting and increasing sparsity levels of a model (supports iterative and one-shot pruning)

All pruning classes inherit from `Pruner` and override the `prune()` method with their own pruning strategy.

D.2 Implementing WANDA for Iterative Pruning and CNNs To adapt WANDA for Iterative Pruning (I.P.), we integrated support for handling 4-D tensors. We computed global importance scores and applied pruning based on a `kthvalue` threshold. Empirical results showed that global WANDA pruning yielded higher accuracy than local WANDA pruning.

D.3 Cubic Scheduler Adapted for Epoch-Wise Pruning The original cubic sparsity scheduler was adapted to compute thresholds and apply pruning at each step, rather than at fixed steps.

$$s(t) = s_{\text{final}} + (s_{\text{init}} - s_{\text{final}})(1 - \frac{t}{t_{\text{total}}})^3$$

D.4 Layers Pruned

- **ResNet-50/101:** The `fc` layer was kept dense. All other layers were pruned.
- **VGG-11/19:** The final `classifier.6` layer was kept dense. All other layers were pruned.
- **ViTs and LLMs:** Only linear layers were pruned, while embeddings and classification heads were kept dense.